

ROS 2 как проверяемая программная система робота

M12 • 12.09.2027 - 14.10.2027 • 75 часов

Результат месяца

Ты объединишь отдельные модели, драйверы и регуляторы в воспроизводимую систему из нескольких узлов. Итоговый проект `telemetry-and-control stack v1.0` включает собственные интерфейсы по необходимости, заданные QoS, разделение обработчиков, преобразования координат с учётом времени, программный имитатор оборудования с жизненным циклом, диагностику и воспроизведение записанного отказа. Это программная основа ног и руки; пока достаточно одной ноги и одного условного пальца в отдельных пространствах имён.

Прими следующую архитектуру: источник данных IMU или сустава публикует данные, оценщик выдаёт состояние, регулятор формирует ограниченный запрос, аппаратный слой исполняет разрешённые команды, монитор следит за свежестью и состоянием, регистратор сохраняет эксперимент. За каждый физический ресурс отвечает один компонент. Управление активируется после проверки готовности, а наличие топика (topic, канала публикации сообщений) в графе не означает готовность датчика или привода.

Среда и входные условия

Основной путь общего плана - ROS 2 Jazzy на Ubuntu 24.04. Поддерживаются пакеты для amd64 и arm64; на MacBook с Apple Silicon используй Linux-среду ARM64. Укажи способ виртуализации, сетевые настройки и доступ к устройствам. Проверь обнаружение участников DDS, передачу USB-устройств в гостевую систему, графику и трассировку небольшим опытом: их работа зависит от виртуальной среды. Если графический интерфейс мешает, сначала запусти систему из консоли и запиши данные, а ограничения графики и физического оборудования укажи отдельно.

Из предыдущих месяцев нужны стабильные поля телеметрии, единицы, монотонное время измерений, модели регулятора и оценки состояния, а также отрицательные тесты. Не переписывай математику при переносе в ROS. Вынеси проверенные вычисления в библиотеку и вызывай её из небольшого узла, чтобы тест без ROS продолжал работать. C++ используй для измерения времени выполнения и аппаратного слоя, Python — для сценариев и анализа; третий язык необязателен.

Что считать завершением

Система собирается и запускается по документации; граф и ответственность за интерфейсы понятны; выбор QoS и режима выполнения обоснован измерениями. Имитатор корректно активируется и деактивируется, а записанный сбой воспроизводится. Подготовь материалы для архитектурной проверки и воспроизведения записи на контрольном этапе G2. Его интеграционные часы находятся вне этих 75 часов. Работа имитатора в ROS не закрывает отдельные критерии HIL с реальным оборудованием. Все неподтверждённые свойства укажи в отчёте.

Как организовать 75 часов

Четыре рабочих блока

Блок А, часы 1–20: граф, сообщения и запуск. Блок В, 21–40: DDS, QoS и обработчики. Блок С, 41–60: TF2, URDF и жизненный цикл аппаратного компонента. Блок D, 61–75: диагностика, запись и воспроизведение. В первых трёх блоках по 10 часов теории и разбора и 10 часов практики; в четвёртом 5 часов разбора и 10 часов практики. Итого $35 + 40 = 75$ часов. Рабочие шаги на страницах 9–10 раскрывают эти же часы, поэтому отдельно прибавлять время на них не нужно.

Каждый блок теории из десяти часов раздели на пять занятий по два часа: 20 минут на повторение предыдущей модели, 45 минут на чтение нужного раздела, 40 минут на вывод небольшого примера или разбор журнала и 15 минут на запись вывода. Десятичасовой воскресный практикум также раздели на пять отрезков по два часа с перерывами между ними. Перерывы не входят в чистые учебные часы. Последние пять часов разбора распредели как $2 + 2 + 1$.

Сначала вопрос, затем инструмент

Основная среда - ROS 2 Jazzy на Ubuntu 24.04, colcon, rclcpp для C++ и rclpy для Python; также нужны rosbag2, ros2_control, tf2 и один инструмент построения графиков. Начиная занятие с проверяемого вопроса: как вход влияет на выход, какое состояние недоступно наблюдению и по каким данным можно судить о его изменении. До запуска запиши ожидаемый знак, масштаб или порядок событий. Сравни прогноз с результатом и разбери хотя бы одно расхождение. График должен сопровождаться объяснением наблюдаемой зависимости.

Веди один репозиторий с каталогами model, config, experiments, tests и reports. У каждого опыта должны быть уникальный идентификатор, версия кода, конфигурация, источник параметров, начальное состояние, единицы и исходные данные. Скрипт анализа читает сохранённый опыт и строит отчёт без ручного исправления измерений. Итоговый проект ROS 2 telemetry-and-control stack v1.0 объединяет телеметрию и управление; отдельные упражнения постепенно составят набор его проверок.

Учёт времени и сокращение объёма

Календарные границы могут совпадать с соседним модулем. Отдых, отпускные интервалы и интеграционные проверки остаются как в общем календарном плане; пронумерованные шаги размещаются в доступное время без двойного учёта часов. Дополнительные часы к этому модулю не приписываются. На входе оцени готовность оборудования и библиотек, а не трать практикум на незапланированную смену платформы.

Если отстаёшь, сократи панель до четырёх графиков, оставь один имитатор оборудования и один способ объединения узлов в процессе. Сохрани таблицу ответственности за интерфейсы, опыт с QoS, измерение задержки обработчиков, TF на момент измерения, проверку жизненного цикла и автоматическое воспроизведение отказа. Второй инструмент визуализации и перенос узлов на Rust необязательны. Для недоступной трассировки или физического драйвера укажи конкретный непроверенный критерий; проверка архитектуры на имитаторе не заменяет HIL.

Граф, интерфейсы и запуск

Изучи виды взаимодействия узлов

Топик подходит для потока IMU или состояния сустава; сервис - для короткого запроса конфигурации или состояния; действие (action) - для длительного движения с обратной связью и отменой. Параметр хранит настройку, но не заменяет поток заданий в каждом цикле. Разбери узлы, топики, сервисы, действия и параметры, затем запуск (launch), пространства имён и объединение узлов в одном процессе (composition). Такое объединение меняет передачу данных и последствия отказа; число логических компонентов не обязано совпадать с числом процессов.

Сначала используй стандартные `sensor_msgs/Imu` и `JointState` там, где они точно передают смысл. Собственное сообщение нужно для полей, которых не хватает описанию интерфейса: идентификатора команды, причины отказа, версии состояния или связанной метки качества. Изучи преобразование `.msg`, `.srv` и `.action` в IDL, массивы и строки ограниченной длины и сериализацию. Для каждой величины укажи единицы и систему координат, для временной метки `timestamp` — источник времени. Версионируй семантические изменения, а не только имена пакетов.

Задача 1. Интерфейсы одной ноги и пальца

Исходные данные: определения из M11, одна нога с известными именами суставов и один программный имитатор пальца. 1) Нарисуй граф из источника, оценителя, контроллера и монитора. 2) Составь таблицу владельцев полей. 3) Создай пакет интерфейсов и два минимальных узла публикации и проверки. 4) Запусти `/leg` и `/hand` в разных пространствах имён. Результат: документ интерфейсов, сообщения и рабочий пример. Проверка: углы в рад, угловая скорость гироскопа в рад/с, ускорение в м/с²; имена суставов сопоставляются по имени, не по случайной позиции массива. Ошибка: при обзоре графа обнаружь двух издателей, которые одновременно задают одну эталонную конфигурацию.

Задача 2. Управляемый запуск и отмена

Создай файл `launch` для двух экземпляров стенда, YAML-параметры и действие для небольшого движения имитируемого сустава. Обработай цель `goal`, обратную связь `feedback`, результат `result` и отмену `cancel`; после отмены имитатор должен перейти в заданное состояние остановки. Проверяй параметры до активации. Сохрани единую команду запуска и сценарий отмены. Проверка: из другого рабочего каталога ресурсы пакета находятся; неверный предел или неизвестное имя сустава отклоняются с понятной причиной. Бесконечный `respawn` не должен скрывать ошибки, а перезапуск - сам возобновлять движение.

Отдельно сравни узлы в разных процессах и компоненты в одном контейнере. На данном этапе измерь корректность имён и данных; ускорение от передачи внутри процесса (intra-process) оцени по измерениям в следующем блоке.

DDS, QoS и выполнение обработчиков

Изучи различие доставки и актуальности

RMW связывает ROS API с промежуточным программным обеспечением; механизм обнаружения DDS находит участников, а домен отделяет графы в выбранной сети. Зафиксируй RMW_IMPLEMENTATION, ROS_DOMAIN_ID и сетевые условия. QoS включает reliability, durability, history и depth, deadline, lifespan и liveliness. Изучи совместимость предлагаемого и запрашиваемого профилей: несовместимые узлы могут существовать в графе и не обмениваться сообщениями. Режим reliable не даёт приложению гарантии свежести команды к сроку.

Для потока датчика можно предпочесть актуальность полной доставке; для команды движения нужны явный номер, возраст и политика повторов. Старую сохранённую команду нельзя случайно повторить новому подписчику из-за transient_local. Событие deadline сообщает нарушение ожидаемой периодичности, но остановку реализует приложение. История глубиной N ограничивает очередь сообщений, но сама по себе не задаёт максимальную задержку. Проверь сочетание политики и реального поведения выбранного RMW.

Задача 3. Профили QoS и потери сообщений

Источник публикует N нумерованных пакетов измерений с известной частотой. Создай три совместимые пары профилей и одну намеренно несовместимую; добавь медленного подписчика и поздний запуск. Запиши отправленные, принятые, дублированные и пропущенные номера, возраст сообщений и события QoS. Результат: таблица «профиль → наблюдаемое поведение». Проверка: пропуски считаются по seq внутри одного источника и сеанса; перезапуск не учитывается как огромный отрицательный скачок счётчика. Проверка ошибки: запись bag с несовместимым QoS регистратора нельзя считать полной.

Задача 4. Обработчики, задерживающие управление

Реализуй быстрый обработчик телеметрии (callback) и искусственно медленную обработку или сервис. Сравни SingleThreadedExecutor, MultiThreadedExecutor с одной взаимно исключающей группой и вариант с разными группами. В C++ измерь начало и конец обработчика, ожидание в очереди и задержку от публикации до начала обработки в единой шкале времени. Сохрани распределения p50, p95, p99, максимум и схему групп. Проверка: дополнительные потоки не обязаны ускорять обработчики одной группы с взаимным исключением; блокирующий запрос из обработчика не должен приводить к незамеченной взаимной блокировке (deadlock).

Повтори корректную конфигурацию с передачей внутри процесса, указав фактически поддерживанный способ передачи сообщения. Сравни число копирований и задержку только в измеренной конфигурации: не считай передачу без копирования (zero-copy) свойством любого обмена в ROS2. Изоляция вычислений помогает, но сама по себе не делает обычную ОС системой жёсткого реального времени.

TF2, URDF/Хасро и жизненный цикл оборудования

Изучи преобразования координат с учётом времени

Опиши дерево `base_link` → `imu_link` и `base_link` → `hip` → `knee` → `foot`; для пальца используй собственные имена. TF2 хранит преобразования с временными метками и отвечает на запрос для выбранного момента. Последнее доступное преобразование может отличаться от преобразования на момент измерения. Разбери статические и подвижные системы координат, буфер, таймаут, экстраполяцию и обработку отсутствующего звена. Неверная система координат или временная метка могут вызвать пространственную ошибку, похожую на ошибку фильтра.

В URDF/Хасро различай `visual`, `collision` и `inertial`. Масса задаётся в кг, центр масс - в метрах, инерция - в кг·м²; проверь положительность и физический смысл тензора. Для изображения `visual` допустима подробная полигональная модель, для столкновений `collision` - упрощённая. Ось и пределы сустава `joint` должны соответствовать CAD и приводу. `ros2_control` предоставляет аппаратные интерфейсы состояния и команд (`state` и `command` interfaces); методы `read` и `write` связывают их с устройством. Жизненные циклы аппаратного компонента и обычного узла связаны архитектурой, но относятся к разным объектам.

Задача 5. Положение стопы на момент измерения

Используй геометрию M5 и тестовую траекторию суставов. 1) Создай Хасро ноги и отдельную систему координат IMU. 2) Публикуй состояния суставов и преобразования дерева. 3) Преобразуй точку стопы в `base_link` для момента измерения, заданного `timestamp`. 4) Сверх результат с ручным расчётом и кинематикой предыдущих месяцев. Сохрани URDF, дерево систем координат и тест координат. Проверка: известная нулевая конфигурация даёт ожидаемые значения в метрах; для позднего измерения запрашивается историческое преобразование. Если преобразование отсутствует или устарело, сформируй диагностируемый отказ, а не нулевую точку.

Задача 6. Имитатор оборудования: запрет ранней команды

Создай минимальный аппаратный компонент `ros2_control` с интерфейсами состояния и команды `position` для выбранных суставов. `Configure` проверяет имена и пределы; `activate` разрешает команду после получения начального состояния; `deactivate` переводит имитатор в заданный режим. `Read` возвращает синтетическую обратную связь, `write` применяет ограничение и обновляет счётчик. Сохрани пакет, конфигурацию `controller manager` и тест переходов. Проверка: до активации движение не выполняется; ошибка `read` или `write` вызывает ожидаемое состояние и событие. Имитатор подтверждает интерфейс и логику; реальный драйвер, задержки и физическую остановку проверяй отдельно.

Диагностика, трассировка и воспроизведение сбоя

Изучи средства наблюдения за программой

Журнал объясняет отдельное событие; диагностический статус описывает состояние компонента; метрика показывает поведение на интервале; трассировка раскрывает порядок и длительность выполнения. Отслеживай актуальность данных IMU и суставов, состояние аппаратного слоя, ошибки регулятора, счётчики пропусков, возраст команды и нагрузку от записи. Каждое предупреждение должно содержать причину, временную метку и условие восстановления. Постоянная строка «ОК» этих сведений не даёт.

Rosbag2 записывает сообщения и сведения об их времени. Параметры, версии кода, начальные значения генератора случайных чисел (seed), аппаратные события и внешние часы сохраняй отдельно. Повторная обработка записи не гарантирует прежнего порядка планирования потоков. Для воспроизводимого опыта зафиксируй начальное состояние, use_sim_time, источник /clock и правила порядка входов. Для событий с одинаковой временной меткой timestamp могут понадобиться собственный seq или детерминированный обработчик в тесте.

Задача 7. Запись, панель и причинная цепочка

Запиши данные IMU, состояния суставов, оценку, команды, diagnostics и нужные TF. Сохрани параметры и перечень версий manifest. В Plotjuggler либо Foxglove создай одну панель: задание и угол, возраст IMU, состояние аппаратного слоя, ошибка регулятора. Затем внеси задержку источника и найди цепочку «датчик устарел → монитор обнаружил → команда заблокирована». Результат: запись bag, конфигурация панели и краткий отчёт. Проверка: виден исходный номер измерения и каждый переход; событие не исчезает при переключении графика. Сравни счётчики источника и записи, чтобы оценить полноту bag.

Задача 8. Автоматическое воспроизведение отказа

Создай тест launch_testing: запусти систему с имитатором, дождись готовности, воспроизведи фиксированную запись bag и проверь событие отказа. Добавь таймаут и завершение всех процессов. Отдельно включи ros2_tracing для одного случая задержки обработчика, если Linux-среда поддерживает выбранный механизм трассировки; укажи ограничения окружения. Проверь код завершения, содержимое результата и состояние системы после теста. Если узел не достиг готовности или завис при завершении, тест должен закончиться ошибкой, а не ожидать бесконечно.

Определи, какая повторяемость требуется: одинаковые диагностические события и значения в допуске либо одинаковая последовательность обработанных сообщений. Не называй запись детерминированной только потому, что файл не изменился. Укажи контролируемые источники недетерминизма и оставшиеся различия задержек по физическим часам.

Требования к системе ROS 2 v1.0

Пакеты и ответственность

robot_interfaces содержит только сообщения, сервисы, действия и описание единиц. robot_description хранит Xacro и полигональные модели. robot_core содержит вычисления без зависимостей от ROS; robot_nodes вызывает их из узлов оценки, регулятора и монитора. robot_hardware предоставляет имитатор оборудования; robot_bringup хранит launch и YAML; robot_tests - интеграционные сценарии; experiments - сведения о версиях manifest и записи. Пакеты можно объединить, сохранив эти границы ответственности и отсутствие циклических зависимостей.

Выбери один поток: mock_joint/imu → estimate → control_request → hardware. Монитор получает независимые признаки свежести и разрешает или запрещает активацию. У интерфейса команд один владелец; диагностическая панель не пишет команды в обход него. Для каждого ребра графа запиши тип сообщения, единицы, QoS, частоту, источник времени и поведение при устаревании. Пространство имён позволяет запустить второй экземпляр руки, не изменяя исходники.

Согласование времени

Внутри процесса длительности измеряются монотонными часами. Временная метка ROS-сообщения нужна для общей временной привязки и TF; при нескольких машинах заяви способ синхронизации или ограничь измерение односторонней задержки одной машиной. Нельзя вычитать часы разных устройств и объявлять разность транспортной задержкой без оценки смещения. Отдельно измеряй период публикации, начало обработчика, его длительность и возраст данных в момент вычисления команды.

Изменение системного времени или /clock не должно незаметно менять работу контроля актуальности данных. Для воспроизведения записи заранее выбери виртуальное время; для аппаратной проверки свежести - определённую устойчивую шкалу времени. При паузе воспроизведения поведение регулятора должно соответствовать выбранному режиму. Бесконечно повторять прошлую физическую команду недопустимо. Проверяй воспроизведение на имитаторе с отдельным пространством имён и конфигурацией.

Воспроизводимость и документация

Одна команда собирает рабочее пространство, другая запускает имитатор, третья выполняет проверки. Укажи зависимости и проверенную ОС и архитектуру, порядок действий в терминалах и ожидаемые топики. Отчёт содержит граф, таблицу QoS, схему жизненного цикла, измерения обработчиков, записанный отказ и ссылку на тест. При сборке из чистого каталога проверяются пути к установленным ресурсам, а не случайные абсолютные пути с твоего компьютера.

Снимки панели пригодны для быстрого обзора, но исходным доказательством остаются bag, manifest и вычисленные показатели. Основа проекта - ясное распределение ответственности: кто создаёт данные, кто принимает решение и кто имеет право задействовать привод.

Шесть испытаний интеграции и отказов

Доставка и исполнение

F1 — несовместимые QoS. Запусти издателя и подписчика с конфликтующими требованиями `reliability` или `durability` и покажи отсутствие ожидаемого обмена. Ожидание: диагностическое объяснение, а не вывод «датчик сломан». Затем исправь профиль и повтори тот же опыт с N пакетами, сохранив счётчики.

F2 - медленный обработчик. Искусственная обработка занимает больше периода источника. Ожидание: очередь и возраст данных растут либо возникают пропуски согласно профилю; измеряются `r99` и максимум. Объясни, как выбор исполнителя и групп обработчиков влияет на результат. Средняя частота, близкая к заданной, не подтверждает актуальность данных.

F3 — перезапуск источника. Источник сбрасывает `seq` и временно пропадает. Ожидание: новый `session_id` отличает перезапуск от переполнения счётчика или перестановки пакетов, контроль актуальности данных обнаруживает паузу, повторное появление данных не приводит к автоматической активации устройства. Состояние восстановления зафиксировано отдельным событием.

Геометрия, аппаратный слой и воспроизведение

F4 — TF из будущего или прошлого и неверная система координат. Подай пакет измерений вне доступного буфера и пакет измерений с неизвестным `frame_id`. Ожидание: причина отказа видна, команда на основании неподтверждённого преобразования не формируется. В исправленном случае известная точка стопы совпадает с независимым расчётом в заявленном допуске.

F5 — ошибка `configure`, `read` или `write`. Подай некорректный предел сустава, затем симулируй аппаратную ошибку активного имитатора. Ожидание: ранняя активация запрещена, причина диагностируется, контроллеры не продолжают молча писать в недействительный интерфейс. Конкретную последовательность действий управляющего и аппаратного компонентов закрепи по установленной версии `ros2_control` и тесту.

F6 — `bag` с пропуском и зависание при завершении. Удали нужный поток или останови один узел до конца сценария. Ожидание: тест воспроизведения выявляет отсутствие события или данных и заканчивается по таймауту; все процессы завершены. Повторный запуск не наследует старые узлы и случайный `/clock` от прошлого теста.

Как подтвердить исправление

Каждый отрицательный тест имеет исправленную пару с теми же начальными условиями. В отчёте запиши внесённую ошибку, наблюдаемое событие, задержку обнаружения, конечное состояние и путь к записи `bag` или трассировке. Если причина ещё не выяснена, статус остаётся открытым. Не подменяй исправление увеличением всех таймаутов: новый предел времени должен быть объяснён измерением и требованиями управления.

Особый случай виртуальной машины: обнаружение участников может зависеть от сетевого режима, а трассировка — от ядра и доступа к системным механизмам. Это ограничение среды не следует скрывать в статусе «всё проверено»; консольные и проверки имитатора всё равно выполняются, для недоступной проверки с оборудованием составь отдельный план повторения.

Часы 1-40: граф и поведение доставки

Часы 1-20: интерфейсы и запуск

Теория 1-10: 1-2 — граф и ответственность; 3-4 — топики, сервисы и действия; 5-6 — параметры и IDL; 7-8 — launch и пространства имён; 9-10 — объединение узлов в процессе и план отмены движения. Это пять занятий по два часа; каждое заканчивай записью интерфейсного решения для ноги и его причины. Не изучай все пакеты ROS одновременно.

Практика 11-20: 11-12 — собери минимальное пространство Jazzu и проверь сеть и архитектуру; 13-14 — выполни задачу 1; 15-16 — подключи узлы к библиотеке M11; 17-18 — выполни задачу 2; 19-20 — запусти два пространства имён и проверь обработку неверных параметров и отмены. Сохрани конфигурацию среды и точные команды, потому что они понадобятся для чистого воспроизведения.

Перед переходом дальше проверь: есть граф, владелец каждого интерфейса, рабочий launch и корректная отмена движения имитатора. Не осталось скрытой зависимости от текущего каталога или случайно запущенного терминала.

Часы 21-40: QoS и исполнитель

Теория 21-30: 21-22 — RMW, DDS, домены и обнаружение; 23-24 — reliability, durability и history; 25-26 — deadline, lifespan, liveliness и совместимость профилей; 27-28 — исполнители и группы обработчиков; 29-30 — определения измеряемых задержек и передача внутри процесса. Запиши прогноз для медленного подписчика до запуска эксперимента.

Практика 31-40: 31-32 — выполни сравнение профилей задачи 3; 33-34 — добавь позднего подписчика и перезапуск; 35-36 — выполни задачу 4; 37-38 — сравни группы, процессы и передачу внутри процесса; 39-40 — построй распределения и выбери профиль каждого потока. Зафиксируй не только выбранный вариант, но и отвергнутую конфигурацию с причиной.

Перед переходом дальше проверь: QoS выбран по требованиям конкретного потока, возраст данных измерен корректно, взаимная блокировка и накопление очереди воспроизводятся. Утверждения о скорости относятся к измеренной ОС и RMW.

Часы 41-75: геометрия, имитатор и запись

Часы 41-60: системы координат и аппаратный слой

Теория 41-50: 41-42 — дерево TF и оси; 43-44 — время, буфер и экстраполяция; 45-46 — URDF/Xacro, массы и геометрия столкновений; 47-48 — интерфейсы состояния и команд `ros2_control`; 49-50 — `configure`, `activate` и `deactivate` и ошибки. В конце каждого занятия проверяй один конкретный факт: знак сустава, метрический размер, временная метка либо право записи команды.

Практика 51-60: 51-52 — собери описание ноги; 53-54 — выполни задачу 5; 55-56 — реализуй минимальный имитатор задачи 6; 57-58 — подключи `controller manager` и проверь переходы; 59-60 — внеси ошибки TF, `read` и `write` и оформи схему. Для этого блока сложная визуальная модель не нужна: геометрически верные простые формы лучше непроверенного детального полигональной модели.

Перед переходом дальше проверь: известная точка стопы проверена численно, имитатор оборудования не исполняет раннюю команду, ошибки проявляются в диагностике. Физическая готовность отдельно не заявляется.

Часы 61-75: наблюдение и повтор отказа

Разбор 61-65: 61-62 — `rosvbag2`, `/clock` и состояние при воспроизведении; 63-64 — диагностика, журналирование и трассировка и критерии `launch_testing`; 65 — составь финальный перечень подтверждающих результатов. Последние пять часов делятся 2 + 2 + 1, а подготовка G2 здесь означает сбор результатов уже выполненных проверок.

Практика 66-75: 66-67 — выполни задачу 7; 68-69 — сохрани и объясни записанный отказ; 70-71 — выполни задачу 8 с таймаутом и завершением процессов; 72-73 — выполни шесть отрицательных сценариев; 74-75 — чистая сборка, воспроизведение и отчёт. Если трассировка недоступна, приложи временные метки обработчиков, описание ограничения среды и инструкцию повторения на подходящей Linux-системе.

Перед переходом дальше проверь: система v1.0 воспроизводится и позволяет объяснить один реальный программный отказ. Материалы готовы для проверки архитектуры и воспроизведения записи на G2; остальные критерии этого контрольного этапа учитываются отдельно.

Приёмка и подготовка к цифровому двойнику

Демонстрация рабочего стека

Из чистого окружения собери пакеты, запусти два пространства имён, покажи поток данных IMU и суставов, а также отмену действия. Включи медленный обработчик и объясни произошедшее по метрикам. Заверши демонстрацию воспроизведением bag с отказом и проверкой, что автоматический тест обнаружил ожидаемое событие. Покажи RViz, счётчики, состояние аппаратного слоя и отсутствие незавершённых процессов.

Обязательная приёмка: сборка и launch воспроизводятся; единицы и ответственность за данные описаны; QoS совместим и проверен; для TF используется время измерения; имитатор корректно обработал отказ и перешёл в предусмотренное состояние. Запись содержит нужные потоки, критерии повторяемости заданы явно, автоматические тесты завершаются и при успехе, и при неудаче. Сам запуск узла ещё не подтверждает корректность исполнения команды.

Самопроверка

1) Когда нужно действие вместо сервиса? 2) Почему параметры не заменяют поток команд? 3) Как несовместимые QoS видны в графе? 4) Почему reliable не означает свежий? 5) Чем deadline отличается от контроля актуальности данных в приложении? 6) Почему несколько потоков не спасают одну группу обработчиков с взаимным исключением?

7) Как получить преобразование на время измерения? 8) Почему visual и collision нужны разные? 9) Кто владеет интерфейсом команд? 10) Что происходит при ошибке read или write именно в твоём стеке? 11) Какие данные вне bag нужны для повторения? 12) Почему одинаковый bag может исполняться с разными задержками? Ответы подкрепи конфигурацией, логом или тестом.

Передача в M13 и граница G2

Передай Хасро ноги, имена суставов, аппаратные интерфейсы, launch, QoS, шаблон bag и каталог отказов. Это каркас для цифрового двойника в Gazebo: модель будет заменять имитатор оборудования, сохраняя внешние интерфейсы и требования к ним. Начальная геометрия квадрупеда в M13 расширит одну проверенную ногу; физический квадрупед ещё не требуется.

Материалы по архитектуре и воспроизведению записи готовы к G2, но общий план также содержит критерий HIL для одной ноги, раскрываемый в M13. До опыта с реальным оборудованием этот критерий остаётся открытым; программный имитатор его не закрывает. Наступление календарного срока G2 не означает, что всем пунктам можно присвоить PASS.

При отставании сократи панель до четырёх графиков и оставь один имитатор оборудования. Сохрани измерения QoS и времени выполнения, TF на момент измерения и воспроизводимый отказ. Если проверка физического интерфейса или трассировка перенесена, укажи окружение, команду, ожидаемый файл и конкретный непроверенный критерий.

Литература: короткий маршрут по разделам

Литература преимущественно на английском; план и определения - на русском. Читай выбранные разделы перед задачей, а не книги целиком. Основное чтение входит в существующие 35 часов разбора; справочник открывай по необходимости.

Основное чтение

Э. Ренар, ROS 2 from Scratch (2024). [Книга и оглавление](#). Гл. 3 Uncovering ROS 2 Core Concepts; в гл. 5–9 только разделы о собственных интерфейсах, механизмах действий, YAML и настройке launch. К задачам 1–2: выпиши требования к потоку, запросу и отменяемому действию. Гл. 10–12 используй точно для TF и URDF.

К. Линч, Ф. Парк, Modern Robotics (2017). [Книга и учебные разделы](#). §3.2.1 Rotation Matrices и §3.3.1 Homogeneous Transformation Matrices. К задаче 5: вручную составь преобразование camera → body → world и обратное; это проверка смысла TF, а не новое прохождение кинематики.

Справочное чтение

М. ван Стен, Э. Таненбаум, Distributed Systems, 3-е изд. (2017). [Авторское оглавление](#). В гл. 3 Processes, 4 Communication и 8 Fault tolerance выбери объяснения потоков, обмена и частичных отказов. К задачам 3–4 и 8: различай доставку, выполнение, своевременность и восстановление; весь учебник не нужен.

Три рабочих справочника

[ROS 2 Design: QoS](#) - DDS Quality of Service policies и QoS profiles; задача 3, таблица совместимости. Значения и фактическое поведение сверяй с Jazzy.

[Jazzy: Using time in tf2](#) - Background и Tasks; задача 5, запрос преобразования на время измерения и обработка его отсутствия.

[ros2_control: Hardware Components](#) - жизненный цикл и интерфейсы состояния и команд; задача 6, один владелец ресурса и разрешение активации.

После чтения ответь по своему стеку: кто владеет командой, когда измерение устаревает и что прекращает воздействие при отказе. Установку, запись и дополнительные инструменты смотри на странице 14.

Термины и определения

Узел (node). Логический компонент ROS с собственными интерфейсами; несколько узлов могут исполняться внутри одного процесса.

Граф ROS (ROS graph). Совокупность узлов и их коммуникационных связей, обнаруживаемых в заданном домене и доступной сети.

Топик (topic). Именованный типизированный поток сообщений; публикация обычно не является подтверждением исполнения команды подписчиком.

Сервис (service). Взаимодействие запрос-ответ для ограниченной операции; длительное движение с отменой удобнее выразить через действие.

Действие (action). Протокол выполнения длительной задачи с обратной связью, результатом и запросом отмены, требующим обработки сервером.

Параметр (parameter). Именованная настройка узла с типом и правилами изменения; не замена последовательности управляющих команд.

Пространство имён (namespace). Префикс имён ROS, позволяющий разделить экземпляры подсистем; сам по себе не обеспечивает сетевую изоляцию.

DDS и RMW. DDS реализует обмен данными; слой RMW связывает ROS с конкретной реализацией промежуточного ПО.

QoS. Набор политик доставки и хранения сообщений; для обмена предложенный и запрошенный профили должны быть совместимы.

Надёжность доставки (reliability). Политика best effort либо reliable; повторная доставка не гарантирует актуальность сообщения к сроку управления.

Доступность сохранённых данных (durability). Политика доступности ранее опубликованных данных позднему подписчику; transient local требует осторожности для команд движения.

Срок обновления (deadline). Ожидаемый максимальный интервал между обновлениями; событие нарушения сообщает проблему, а реакцию реализует приложение.

Исполнитель (executor). Механизм, выбирающий готовые обработчики и исполняющий их в доступных потоках согласно конфигурации групп.

Группа обработчиков (callback group). Группа функций обратного вызова с правилом взаимного исключения либо возможностью одновременного исполнения.

Взаимная блокировка (deadlock). Состояние взаимного ожидания ресурсов или событий, при котором участники не могут продолжить выполнение.

Метка времени (timestamp). Численное время события в указанной шкале; время измерения может отличаться от времени получения сообщения.

TF2. Буферизованное дерево преобразований координат, позволяющее запрашивать связь систем на заданный момент времени.

Жизненный цикл (lifecycle). Явные состояния настройки, активации, деактивации и ошибки компонента с разрешёнными переходами между ними.

Аппаратный интерфейс (hardware interface). Контракт чтения состояния и записи команды ресурса; право управления должно принадлежать определённому владельцу.

Воспроизведение записи (replay). Повторная подача записанных данных с известными временем и конфигурацией; исходный порядок вычислений может не воспроизводиться.

Что купить и установить в этом месяце

ИСПОЛЬЗОВАТЬ ИМЕЮЩЕЕСЯ

Используй MacBook Pro M4, установленную UTM и Ubuntu 24.04 ARM64 из M1, Git, CMake, C++ и Python. Для M12 достаточно программного имитатора оборудования: новый бортовой компьютер, мотор или датчик покупать не нужно. Платы и измерительные инструменты M4–M7 понадобятся позже для HIL.

УСТАНОВИТЬ СЕЙЧАС

В гостевой Ubuntu установи ROS 2 Jazzy Desktop с RViz, colcon, rosdep, vcstool, ros2_control и ros2_controllers, tf2, robot_state_publisher, Xacro и rosbag2. Основа: [официальная установка Jazzy](#). Сохрани архитектуру arm64, версии пакетов, RMW и ROS_DOMAIN_ID; не смешивай дистрибутивы в одной оболочке или рабочем пространстве.

Сначала выполни [RRBot demo ros2_control](#) с имитатором оборудования. Библиотеку регулятора на C++ или Python сохрани независимой от ROS. Для пакета интерфейсов держи под рукой [IDL: Type system](#); размеры массивов, строки и единицы определи до сборки.

ПО НЕОБХОДИМОСТИ

Для задач диагностики используй [Plotjuggler](#) или существующий графический анализатор. [ros2_tracing](#) установи при поддержке выбранного механизма в Linux и наличии прав; иначе сохрани измерения обработчиков и укажи ограничение трассировки. [launch_testing](#) добавь для запуска процессов с таймаутом.

ПРОВЕРКА УСТАНОВКИ

1) Собери два минимальных узла через colcon, проверь работу talker и listener, а также обнаружение в выбранной сети. 2) Открой RViz с TF ноги; графику UTM проверь фактически. 3) Активируй и деактивируй имитатор оборудования без самопуска. 4) Запиши и воспроизведи [rosbag2](#) с timestamp и seq, убедись в совместимости QoS. Передачу USB-устройств в гостевую систему проверь до M13, а не считай свойством любой виртуальной машины.

Подготовка и эти проверки заменяют повторную настройку и чтение внутри текущих 75 часов. Наличие страницы установки не означает, что программы уже установлены. Лицензионную книгу покупать необязательно, если есть доступ; аппаратных закупок на этот месяц нет.

Основы движения: физический смысл URDF

Обязательно: четыре описания одного звена

Какие моменты потребуются ноге, изображённой в RViz? Кинематика задаёт связи, оси и преобразования; `visual` - изображение; `collision` - геометрию столкновений и контакта; `inertial` - массу, центр масс и инерцию. Перед работой преобразуй одну точку между двумя системами координат и объясни различие кг и кг·м².

`robot_state_publisher` вычисляет TF по геометрии и положениям суставов, не интегрируя действие сил. `ros2_control` задаёт интерфейсы команды и состояния. Имя `effort` не превращает позиционный сервопривод в управляемый источник момента.

Один брусок: геометрия и инерция из размеров

База B закреплена, система правая: x вперёд, z вверх. Сустав в начале координат вращается вокруг +y. При $q = 0$ звено направлено по +x, положительный q опускает конец: **p_tip в B = $(L \cos q, 0, -L \sin q)$** . При $L = 0,20$ м: $q = 0$ даёт (0,20; 0; 0) м, $q = \pi/2$ даёт (0; 0; -0,20) м. Масса сюда не входит.

Однородный брусок: длина $L = 0,20$ м вдоль x звена, ширина и высота $d = 0,02$ м, $m = 0,30$ кг. Начало системы координат звена находится в суставе; центр масс в этой системе — (0,10; 0; 0) м. Систему координат инерции помести в центр масс, сохранив параллельность осей:

$$I_{xx} = m(d^2 + d^2)/12 = 0,000020 \text{ кг}\cdot\text{м}^2;$$

$$I_{yy} = I_{zz} = m(L^2 + d^2)/12 = 0,001010 \text{ кг}\cdot\text{м}^2.$$

Внедиагональные элементы нулевые по симметрии. В `inertial` задай этот тензор относительно COM и origin центра масс; `visual` и `collision` бруска тоже смести на $L/2$.

Инерция относительно оси сустава: **$I_{joint} = I_{yy} + m(L/2)^2 = 0,004010 \text{ кг}\cdot\text{м}^2$** . Добавка учитывает движение COM по окружности. Подстановка I_{joint} вместо I_{yy} при origin в COM повторно учтёт перенос и исказит динамику.

При $q = 0$ сила тяжести (0; 0; -mg) Н приложена в $x = 0,10$ м. Её момент вокруг +y: $+mgL/2 = +0,2943 \text{ Н}\cdot\text{м}$; для удержания привод даёт -0,2943 Н·м. При $q = \pi/2$ плечо силы нулевое. Знак следует из $r \times f$.

Удвоение массы при той же форме удваивает инерцию и гравитационный момент; TF при заданном q прежний. Изменение `visual` меняет изображение. Увеличение геометрии `collision` меняет контакт и зазоры, но не доказывает прочность звена. Брусок проверяет единицы; реальные массы и распределение материала берутся из измерений или CAD.

Две проверки модели в часах 41-60

1. Выдели в описании ноги проверочное звено; сохрани URDF и `p_tip` из ручного FK и TF при $q = 0$ и $\pi/2$. Учебный допуск — 10^{-6} м для этой модели. **2.** В копии удвой массу; скриптом проверь удвоение I и момента, неизменность FK. Сохрани массу, COM, шесть элементов I и системы координат; главные моменты инерции должны быть положительными.

Время: 1 час теории из 45-46 (URDF, массы, геометрия столкновений), по 1 часу практики из 51-52 (описание ноги) и 53-54 (задача 5, TF). Это проверки существующей модели внутри 75 часов. Контакт — в M13; реальных команд приводу здесь нет.

Читай: [Lynch, Park, Modern Robotics §8.2, часть 1](#) — COM и инерция; [ROS urdf tutorial, 07-physics.urdf](#) — структура `visual`, `collision` и `inertial`. Условные инерции из XML не копируй: здесь они рассчитаны.